



Statistical Computing in Functional Data Analysis: The R Package `fda.usc`

Manuel Febrero-Bande

Universidad de Santiago
de Compostela

Manuel Oviedo de la Fuente

Universidad de Santiago
de Compostela

Abstract

This paper is devoted to the R package **fda.usc** which includes some utilities for functional data analysis. This package carries out exploratory and descriptive analysis of functional data analyzing its most important features such as depth measurements or functional outliers detection, among others. The R package **fda.usc** also includes functions to compute functional regression models, with a scalar response and a functional explanatory data via non-parametric functional regression, basis representation or functional principal components analysis. There are natural extensions such as functional linear models and semi-functional partial linear models, which allow non-functional covariates and factors and make predictions. The functions of this package complement and incorporate the two main references of functional data analysis: The R package **fda** and the functions implemented by [Ferraty and Vieu \(2006\)](#).

Keywords: functional data regression, representation of functional data, non-parametric kernel estimation, depth measures, outlier.

1. Introduction

The technological progress has led to the development of new, quick and accurate measurement procedures. As a consequence, further possibilities for obtaining experimental data are now available and some classical paradigms must be revised. For example, it is now possible (and even frequent) to find problems where the number of data points is greater than the number of variables. In many areas it is common to work with large databases, which often increase these observations of a random variable taken over a continuous interval (or in increasingly larger discretizations of a continuous interval).

For example, in fields such as spectroscopy, the measurement result is a curve that, at least,

has been evaluated in 100 points. This type of data, which we call functional data arise naturally in many disciplines. In economics, one could consider curves intra-day stock quotes. In environmental studies, one could find continuous measurements of atmospheric monitoring networks. The importance of functional data in image recognition or spatio-temporal information is also well-known.

Undoubtedly, package **fda** (Ramsay, Wickham, Graves, and Hooker 2012) is a basic reference to work in R programming environment (R Development Core Team 2012) with functional data. The books by Ramsay and Silverman (2005) and Ramsay and Silverman (2002) are well-known references in this field. In both cases all the techniques included are restricted to the space of $\mathcal{L}_2$ functions (the Hilbert space of all square integrable functions over a certain interval). The book by Ferraty and Vieu (2006) is another important reference incorporating non-parametric approaches as well as the use of other theoretical tools such as semi-norms and small ball probabilities that allow us to deal with normed or metric spaces. These authors are part of the French group STAPH maintaining the page <http://www.lsp.ups-tlse.fr/staph> where R software can be downloaded.

Other R software provides implementation of functional data analysis (FDA) in different areas. The package **rainbow** (Shang and Hyndman 2012) for functional data representation, the package **ftsa** (Hyndman and Shang 2012) for functional time series analysis, the package **refund** (Crainiceanu, Reiss, Goldsmith, Huang, Huo, and Scheipl 2012) for functional penalized regression, the package **fpca** (Peng and Paul 2011) implements the restricted maximum likelihood estimation for functional principal components analysis (FPCA), the package **MFDF** (Dou 2009) for modeling functional data in finance via generalized linear models (GLM), the package **fdaMixed** (Markussen 2011), for FDA in a mixed model framework and the package **geofd** (Giraldo, Delicado, and Mateu 2010) for spatial prediction for function value data. In MATLAB **PACE** package (see <http://anson.ucdavis.edu/~ntyang/PACE/>) provides implementation of various methods using FPCA as generalized functional linear models (GFLM) for sparsely sampled functional data, this package is also in R (Liu 2011). Finally, from a Bayesian perspective, Crainiceanu and Goldsmith (2010) have developed tools for GFLM using **WinBUGS**.

The aim of the package **fda.usc** is to provide a broader, flexible tool for the analysis of functional data. Therefore, we propose an integration of the work on non-parametric functional methods implemented by Ferraty and Vieu (2006) with those from the group of the University of Santiago de Compostela (USC), thus complementing and extending some of the functions of package **fda**. The **fda.usc** package is available from the Comprehensive R Archive Network at <http://CRAN.R-project.org/package=fda.usc> and it has the greedy objective of being an integrate framework for all the functional data methods available.

In Section 2.1 we introduce a new R class for functional data: “**fdata**”. Section 2.2 describes the functional representation of an object of class “**fdata**” by basis representation or kernel smoothing. Section 2.3 is focused in normed and semi-normed functional spaces and in how the metrics and semimetrics can be used as measures of proximity between functional data. Section 2.4 is concerned with the concept of statistical depth for functional data see Cuevas, Febrero-Bande, and Fraiman (2007). These depth measures are useful to define location and dispersion measures, for classification and for outlier detection (Febrero-Bande, Galeano, and González-Manteiga 2008) (Section 2.6). Section 3 is devoted to functional regression models with scalar response from different perspectives: Parametric, using basis representation (Section 3.1, 3.2 and 3.3), non-parametric, using kernel smoothing (Section 3.4) and semi-

parametric which combines both (Section 3.5). The Section 3.6 summarizes the functional regression models using a classical example to show the capabilities of the package **fda.usc**. Finally, Section 4 discusses the most important features of the package.

2. Functional data: Definition and descriptive analysis

After a brief introduction of the state of the art in functional data, we describe the most important features, procedures and utilities of the new package. This article is not intended to enter into many methodological details because most of the functions of the package are adaptations of original functions studied in previous publications.

2.1. Functional data definition: The new R class “fdata”

A functional variable $\mathcal{X}$ is just a random variable taking values in a functional space $\mathcal{E}$. Thus, a functional data set is just a sample $\{\mathcal{X}_1, \dots, \mathcal{X}_n\}$ (also denoted $X_1(t), \dots, X_n(t)$ when convenient) drawn from a functional variable $\mathcal{X}$. Usually, $\mathcal{E}$ is assumed to be a normed or semi-normed metric space. If $\mathcal{E}$ is a Hilbert space, functional data can be represented using a basis. This is the approach of the **fda** package for its methodological developments and it is also used by **refund**, **geofd** or **MFDF** packages, among others. In practice, this representation projects an infinite dimensional space into a finite one and has the drawback of the approximation of the original data. However, the functional space $\mathcal{E}$ may not be Hilbertian and in this case, a more flexible representation of functional data is needed. For this reason, this package introduces the new class “fdata”. This new class uses only the values evaluated on a grid of discretization points $\{t_1, \dots, t_m\}$ that can be observed in a non-equispaced way. The drawback now is that every calculation (distance, norms,...) must be made through numerical approximations and the design or the density of this grid could affect the accuracy of the calculations. If functional data is sparsely sampled or there are many missing data points, probably the representation in a basis is mandatory and some other alternatives, as for example **PACE** package, should be considered.

The **fda.usc** package defines an object called **fdata** as a list of the following components:

- **data**: Typically a matrix of (n, m) dimension which contains a set of n curves discretized in m points or **argvals**.
- **argvals**: Locations of the discretization points, by default: $\{t_1 = 1, \dots, t_m = m\}$.
- **rangeval**: Range of discretization points, by default: **range(argvals)**.
- **names**: Optional list with three components: **main**, an overall title, **xlab**, a title for the x axis and **ylab**, a title for the y axis.

All the methods in the package can work with the new class “fdata”. Some basic operations to handle this new class are introduced. For example, generic methods for the *math* group **abs**, **sqrt**, **round**, **exp**, **log**, **cumsum**; for the *OPS* group “+”, “-”, “*”, “/”, “==”, “<”, “>”, “<=”, “>=”; the for *summary* group **all**, **any**, **sum**, **min**, **max**; and other operations **is.fdata()**, **c()**, **dim()**, **ncol()**, **nrow()**, **plot()**, **inprod.fdata()** and **norm.fdata()**. To calculate the derivative of functional data the **fdata.deriv()** function has been implemented, which computes the derivative by different methods, some purely numerical and others based on a

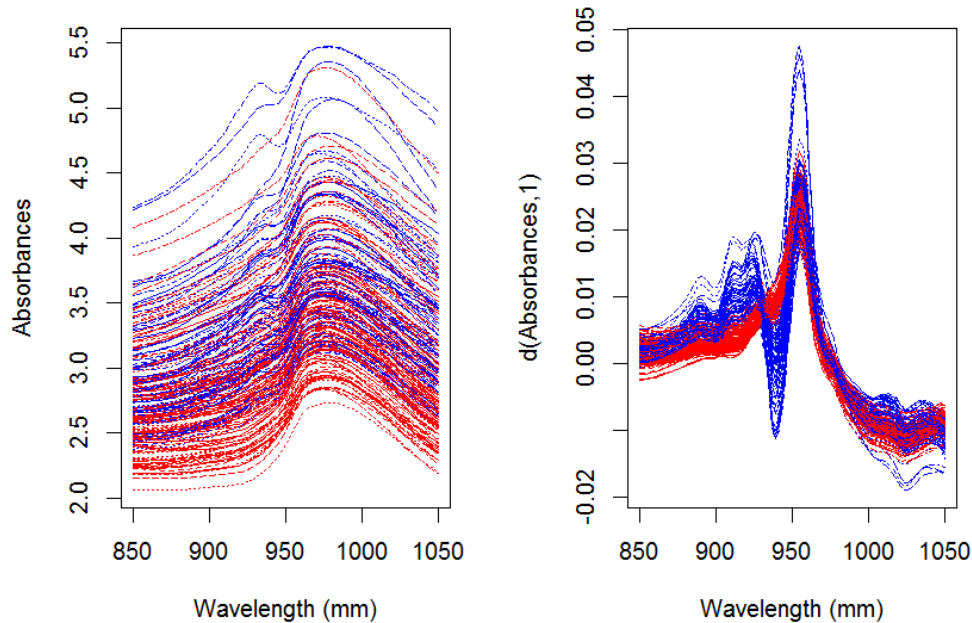


Figure 1: Spectrometric curves (left panel): curves with Fat < 20% (red lines) and curves with Fat $\geq$ 20% (blue lines). Corresponding spectrometric curves after first order differencing (right panel).

basis representation. The choice of the right method depends on the nature of the data and on the grid of discretization.

The function `fdata()` converts a data object of class: “`fd`”, “`fds`”, “`fts`”, “`sfts`” or other formats (“`vector`”, “`matrix`”, “`data.frame`”) to an object of class “`fdata`”. We have included in the “`fdata`” format some popular datasets of functional data: `tecator`, `phoneme`, `poblenou` and `aemet`. As an illustration we applied some of above methods to the `tecator` dataset (`data("tecator")`) that has two components. The first component, `tecator$absorp.fdata`, includes the curves of absorbance for the analysis of pieces of meat stored in the format of class “`fdata`”. Each curve consists of a $m=100$ -channel absorbance spectrum measured along the discretized points: $\text{argvals} = \{t_1 = 850, \dots, t_m = 1050\}$. The second component, `tecator$y`, is a dataframe including `Fat`, `Water` and `Protein` contents of each spectrometric curve obtained by an analytical chemical processing. The `tecator` dataset presents interesting features and it is a well known example in functional data analysis (available at <http://lib.stat.cmu.edu/datasets/tecator>). The following code displays the absorbance curves colored by the fat content, see Figure 1.

```
R> library("fda.usc")
R> data("tecator")
R> names(tecator)
```

```
[1] "absorp.fdata" "y"
```

```

R> absorp <- tecator$absorp.fdata
R> Fat20 <- ifelse(tecator$y$Fat < 20, 0, 1) * 2 + 2
R> plot(tecator$absorp.fdata, col = Fat20)
R> absorp.d1 <- fdata.deriv(absorp, nderiv = 1)
R> plot(absorp.d1, col = Fat20)

```

The representation of a functional dataset should be consistent with the physical interpretation of the phenomenon being described and so it is important to choose the space where the data must be considered. For example, in left panel of Figure 1, the spectrometric curves are plotted showing with different colors two levels of fat content. This representation which implicitly assumes a $\mathcal{L}_2$ space, is not related with the information of fat content. In other words, the vertical shift of these curves has no special relation with the fat content. So, if we are interested in the relationship between the spectrometric curve and the fat content, probably another choice of functional space would be more advisable as shown, for example, in the right panel of Figure 1 with the first derivative of the spectrometric curves.

2.2. Smoothing

If we assume that our functional data $Y(t)$ is observed through the model: $Y(t_i) = X(t_i) + \epsilon(t_i)$ where the residuals $\epsilon(t)$ are independent from $X(t)$, we can get back the original signal $X(t)$ using a linear smoother,

$$\hat{x} = \sum_{i=1}^n s_{ij} y_i \text{ or } \hat{x} = S y$$

where s_{ij} is the weight that the point t_j gives to the point t_i and $y_i = Y(t_i)$. In the package two procedures have been implemented for this purpose. The first one is the representation in a $\mathcal{L}_2$ basis (or penalized basis) and the second one is based on the smoothing kernel methods.

Basis representation

A curve can be represented by a basis when the data is assumed to belong to $\mathcal{L}_2$ space. A basis is a set of known functions $\{\phi_k\}_{k \in \mathbb{N}}$ that any function could be arbitrarily approximated by a linear combination of a sufficiently large number k_n of these functions, see [Ramsay and Silverman \(2005, p. 43 and 44\)](#).

The procedure recovers the function $X(t)$ by using a fixed truncated basis expansion in terms of k_n known basis elements,

$$X(t) = \sum_{k \in \mathbb{N}} c_k \phi_k(t) \approx \sum_{k=1}^{k_n} c_k \phi_k(t) = \mathbf{c}^\top \Phi(\mathbf{t}) \quad (1)$$

The projection (or smoothing) matrix is given by: $S = \Phi(\Phi^\top \Phi)^{-1} \Phi^\top$, with degrees of freedom of the fit $df = \text{tr}(S_\nu) = k_n$.

If smoothing penalization is required, a parameter λ will also be provided and in this case the projection matrix S is: $S = \Phi(\Phi^\top \Phi + \lambda R)^{-1} \Phi^\top$, where R is the penalization matrix. Typically R penalizes the integral of the square of the derivative of order 2.

There are several different types of basis (Fourier, B-spline, Exponential, power, polynomial, ...) that can be created using the function `create.fdata.basis()` or directly through the `create.basis-name.basis` from the **fda** package. This package also includes the basis functions based on the data: `create.pc.basis()` and `create.pls.basis()` that provides a functional principal component and partial least square basis, respectively. The choice of the basis should be based on the objective of the analysis and on the data. For example, it is common to use the Fourier basis for periodic data, and B-spline basis for non-recurrent data. Also, the function `fdata2fd()` converts an object of class “fdata” into an object of class “fd” using the basis representation shown in (1).

```
R> class(absorp.fd <- fdata2fd(absorp, type.basis= "fourier", nbasis= 15))
```

```
[1] "fd"
```

```
R> class(absorp.fdata <- fdata(absorp.fd))
```

```
[1] "fdata"
```

Kernel smoothing

The non-parametric methodology and in particular, the kernel smoothing method, can also be used to represent functional data. Now, the non-parametric smoothing of functional data is given by the smoothing matrix $S = (s_{ij})$. For example, for the Nadaraya-Watson estimator (function `S.NW()`): $s_j(t_i) = K\left(\frac{t_i - t_j}{h}\right) / \sum_{k=1}^m K\left(\frac{t_i - t_k}{h}\right)$ where $K(\cdot)$ is the kernel function. Other possibilities for S are the k nearest neighbors estimator (function `S.KNN()`) or the local linear regression estimator (function `S.LLR()`), see Wasserman (2006), Ferraty and Vieu (2006) for further details. Many different types of kernels are considered in the package: Gaussian, Epanechnikov, triweight, uniform, cosine or any user-defined.

Validation criterion

The choice of the smoothing parameter is crucial and, in principle, there is no universal rule that would enable an optimal choice. Among the different selection criterion to select the parameter ν , we have implemented two: Cross-validation (CV) and generalized cross-validation (GCV).

$$\text{Cross-validation : } CV(\nu) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{y}_{-i}^\nu \right)^2 w_i \quad (2)$$

where $\hat{y}_{-i}^\nu$ indicates the estimator based on leaving out the i pair (t_i, y_i) and w_i is the weight at point t_i . This criterion is implemented by the function `CV.S()`.

$$\text{Generalized cross-validation : } GCV(\nu) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{y}_i^\nu \right)^2 w_i \Xi(\nu) \quad (3)$$

where $\Xi(\nu)$ denotes the type of penalizing function.

Generalized cross-validation criterion is implemented in `GCV.S()` function with the following types of penalized functions (see Härdle 1990, for further details):

- Generalized cross-validation (GCV): $\Xi(\nu) = (1 - \text{tr}(S_\nu)n^{-1})^{-2}$.
- Akaike's information criterion (AIC): $\Xi(\nu) = \exp(2\text{tr}(S_\nu)n^{-1})$.
- Finite prediction error (FPE): $\Xi(\nu) = (1 + \text{tr}(S_\nu)n^{-1})/(1 - \text{tr}(S_\nu)n^{-1})$.
- Shibata's model selector (Shibata): $\Xi(\nu) = (1 + 2\text{tr}(S_\nu)n^{-1})$.
- Rice's bandwidth selector (Rice): $\Xi(\nu) = (1 - 2\text{tr}(S_\nu)n^{-1})^{-1}$.

*Smoothing FDA with **fda.usc** package*

Let us now consider, for illustration purposes, the Phoneme dataset `data("phoneme")`¹ see, e.g., [Ferraty and Vieu \(2006\)](#). The `phoneme$classlearn` object contains 250 speech frames with class membership: “sh” (1), “iy” (2), “dcl” (3), “aa” (4) and “ao” (5). From each speech frame, a log-periodogram of length 150 has been stored in `phoneme$learn` which is used as learning sample. The goal is to predict the class membership `phoneme$classtest` using the test sample `phoneme$test`.

The purpose of the function `min.basis()` is to represent the functional data in terms of a (truncated) expansion with respect to a given basis of functions in the corresponding space. Such expansions depend on a parameter $\nu = (k_n, \lambda)$, where k_n is the number of basis elements used in the truncated expansion and λ is the penalization parameter. In the following example (see below R code), `phoneme$learn` is smoothed using `min.basis()` function.

```
R> data("phoneme")
R> learn <- phoneme$learn
R> l <- c(0, 2^seq(-2, 9, length.out = 30))
R> nb <- seq(7, 31, by = 2)
R> out0 <- min.basis(learn, lambda = l, numbasis = nb)
```

The minimum GCV (GCV.OPT=2.3815) is achieved with
the number of basis (numbasis.opt=27)
and lambda value (lambda.opt=81.27995)

In the left panel of Figure 2, the GCV criterion is drawn in function of the number of B-spline basis elements $\nu_1 = \text{nbasis}$ and the penalizing parameter $\nu_2 = \lambda$ using `min.basis()` function and the smoothed functional data $\hat{x}(t)$ (in `out0$fdata.est` object) can be used as the signal we have tried to recover. Figure 3 shows the 11th curve of speech frame x_{11} (`phoneme$learn[11]`) and the smooth representation of the curve $\hat{x}_{11}$ (`out0$fdata.est[11]`). The `min.np()` function returns the “optimal” value (in `out1$h.opt`) of the smoothing parameter $\nu = h$ that best represents the functional data for a range of values of bandwidth h using the validation criterion (2) and (3) shown above. As an illustration, in the right panel of Figure 2, the GCV criterion is drawn in function of the bandwidth h using the function `min.np()` and Figure 3 shows smooth representation of 11th curve of `phoneme$learn` by Nadaraya-Watson (`S.NW()`), local linear regression (`S.LLR()`) method and K nearest neighbors (`S.KNN()`) method.

¹This dataset includes the changes introduced in the file <http://www.math.univ-toulouse.fr/staph/npfda/npfda-phondiscRS.txt>.

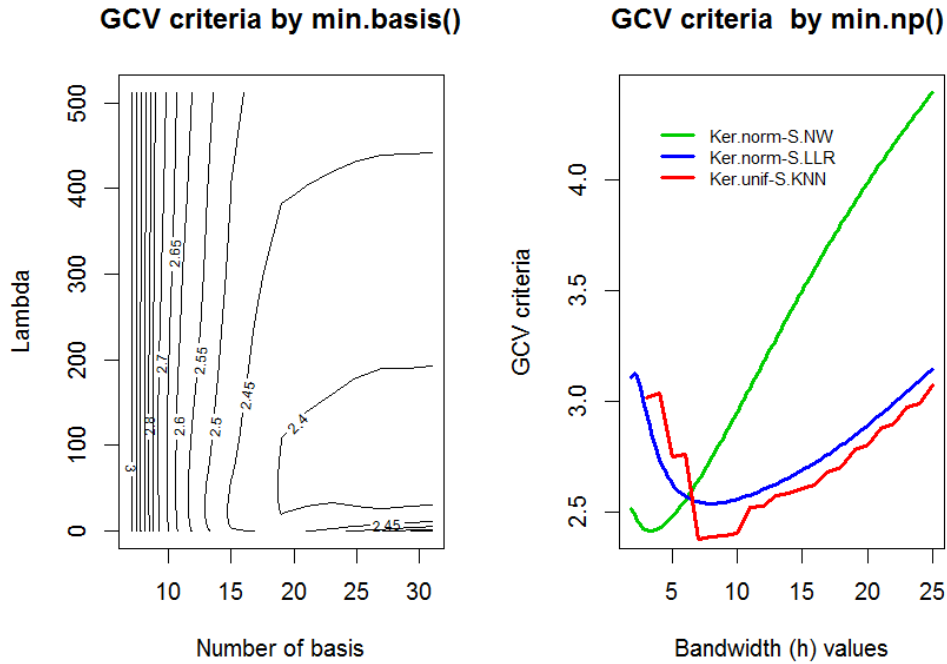


Figure 2: GCV criterion as a function of the number of B-spline basis elements and the penalizing parameter λ (left panel). GCV criterion as a function of bandwidth parameter (right panel): Normal kernel and local linear smoothing matrix (blue line); Normal kernel and Nadaraya-Watson smoothing matrix (green line) and uniform kernel and K nearest neighbors smoothing matrix (red line).

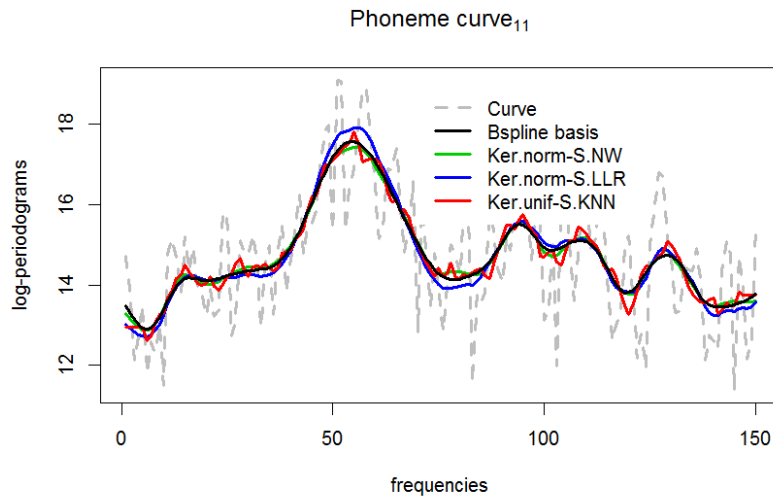


Figure 3: Eleventh phoneme curve: Observed (grey dashed line), smoothed by 27 B-spline basis elements and $\lambda = 81.28$ (black line), smoothed by normal kernel and Nadaraya-Watson smoothing matrix with bandwidth $h = 3.28$ (green line), smoothed by normal kernel and local linear smoothing matrix with bandwidth $h = 7.85$ (blue line) and smoothed by uniform kernel and K nearest neighbors smoothing matrix with bandwidth $h = 7$ (red line).


```
R> out1 <- min.np(learn, type.S = S.NW, par.CV = list(criteria = "GCV"))
```

The minimum GCV (GCV.OPT=2.4119) is achieved with
the h value (h.opt=3.2765)

```
R> out2 <- min.np(learn, type.S = S.LLR, par.CV = list(criteria = "GCV"))
```

The minimum GCV (GCV.OPT=2.5363) is achieved with
the h value (h.opt=7.9482)

```
R> out3 <- min.np(learn, type.S = S.KNN, h = 3:25, Ker = Ker.unif)
```

The minimum GCV (GCV.OPT=2.3746) is achieved with
the h value (h.opt=7)

2.3. Measuring distances

Sometimes, it is difficult to find the best plot to summarize a particular functional dataset because the shape of the graphics depends strongly on the chosen proximity measure. As shown in Figure 1, the plot of $X(t)$ against t is not necessarily the most informative and perhaps, considering another distance between curves we can obtain better results. This package collects several metric and semi-metric functions which allow us to extract as much information possible from the functional variable.

The most general spaces for functional data are the complete metric spaces where only the notion of distance between elements of the space is given. If the metric $d(\cdot, \cdot)$ is associated with a norm (so that $d(X(t), Y(t)) = \|X(t) - Y(t)\|$) we have a normed space (or a Banach space). In some important cases the norm $\|\cdot\|$ is associated with an inner product $\langle \cdot, \cdot \rangle$ in the sense that $\|X\| = \langle X, X \rangle^{1/2}$. A complete normed space (Banach space) whose norm derives from an inner product is called a Hilbert space. The best known example is the space $\mathcal{L}_2[a, b]$ of real square-integrable functions defined on $[a, b]$ with $\langle f, g \rangle = \int_a^b fg$.

Utilities for computing distances, norms and inner products are included in the package. If we focused on $\mathcal{L}_p$ spaces (the set of functions whose absolute value raised to the p -th power has finite integral), the function `metric.lp()` uses Simpson's rule to compute distances between

elements: $\|f(t) - g(t)\|^p = \left(\frac{1}{\int_a^b w(t)dt} \int_a^b |f(t) - g(t)|^p w(t)dt \right)^{1/p}$ where w are the weight and the observed points on each curve t are equally spaced (by default) or not. `norm.fdata()` computes the norm and, specifically for $\mathcal{L}_2$, `inprod.fdata()` calculates the inner product between elements of the space.

In the next example, the distances between some training sample curves of phoneme data (`phoneme$learn`) are calculated by the `metric.lp()` function. Figure 4 shows the dendrogram for a selection of 11 curves of class (3) (corresponding to the index from 110 to 120 curves) and 11 curves of class (5) (from 220 to 230). This example can be understood as a classification problem in which the goal is to classify the curves in 2 classes.

```
R> mdist <- metric.lp(learn)
```